Distributed Message Queue

Setting Up Monitoring and Alerting Capabilities

Ad-Hoc Event Aggregation

Hybrid Real-time System

Distributed Tracing

Ad-Hoc Metric Storage

Data Warehousing Leveraging

Payment System

Digital Wallet

Stock Exchange

Ride-sharing Ecosystem

27
Payment System

In this chapter, we design a payment system. E-commerce has exploded in popularity across the world in recent years. What makes many transactions possible is a payment system behind the scenes that moves money, checks, credits, and debits funds and ensures that

What is a payment system? According to Wikipedia, "A payment system is any system used to settle financial transactions through the transfer of monetary sums. This term includes banks, financial institutions, financial instruments, payment networks, procedures, and technologies that allow for exchange purposes" [1].

Payment system is a term that is understood on the surface but is also intimidating for many developers to work on. It sets off crucial alarms in your organization because bad code and security vulnerability always come to the last minute. In this chapter, we demystify payment systems.

Step 1 - Understand the Problem and Establish Design Scope

A payment system can mean very different things on different people. Some may think it's a digital wallet like Apple Pay or Google Pay. Others may think it's a payment system that handles payments such as PayPal or Stripe. It is very important to determine the exact requirements at the beginning of the interview. Ask yourself the following questions to ensure you are on the right track:

Candidate: What kind of payment system are we building?
Interviewer: Imagine you are building a payment backend for an e-commerce application like Amazon.com. When a customer places an order, many transactions between different backend services are triggered to fulfill the order.
Candidate: What payment options are supported? Credit cards, PayPal, bank wires, etc?
Interviewer: Credit card payment should support all of these options in real life. However, in this interview, we can under-credit card payment as an simplicity.
Candidate: Do we handle credit card payment processing ourself?
Interviewer: No, we don't want to deal with credit card processing, storing, and storing sensitive data, Stripe, Square, etc.
Candidate: Do we need to deal with data or analytics?
Interviewer: Due to extremely high security and compliance requirements, we do not store credit card numbers directly on our system. We rely on third-party payment processing handles sensitive credit card data.
Candidate: Is the application global? Do we need to support different currencies and international payments?
Interviewer: Yes, that is a big part of the challenge. Supporting payments from different countries is one of the most hard challenges.
Candidate: How many payment transactions per second (TPS) do we need to support?
Interviewer: 1 million transactions per day.
Candidate: Do we need to support a mobile app or website, which an e-commerce site like Amazon.com to pay sellers every month?
Interviewer: Yes, we need to support that.
Candidate: I think more details are needed about the requirements, is there anything else critical-path objectives left?
Interviewer: Yes, a payment system includes lots of and interface services (accounting, analytics, etc) and external services (payment processing, risk control, etc). In this interview, we focus on the critical-path services to place a payment order. We need to perform reconciliation and be eventually consistent. This is a strong requirement.

With these questions, we get a clear picture of both the functional and non-functional requirements. In this interview, our focus on developing a payment system that supports the following:

Functional requirements

- Pay-in flow: payment system receives instructions from customers on behalf of sellers.
- Pay-out flow: payment system remits funds to sellers around the world.

Non-functional requirements

- Reliability and fault tolerance. Failed payments need to be retried with a high success rate.
- A reconciliation process between internal services (payment ledgers, accounting systems) and external services (payment service providers) is required. The process ensures/monitors/ verifies that the payment information across these systems is consistent.

Back-of-the-envelope estimation

The system needs to process 1 million transactions per day, which is 1,000,000 transactions / 10^5 seconds = 10 transactions per second (TPS). 10 TPS is a very small TPS for a typical deployed, enterprise-level system. The focus of the system design interview is on how to correctly handle payment transactions, rather than aiming for high throughput.

Step 2 - Propose High-Level Design and Get Buy-In

At a high level, the payment flow is broken down into two steps to reflect money flows:

- Pay-in flow
- Pay-out flow

Take this e-commerce site, Amazon, as an example. When a buyer places an order, the money flows into Amazon's bank account, which is the pay-in flow. Although the money is in Amazon's bank account, it does not belong to all of the money. The seller owns a substantial portion of the money. Amazon keeps a record of how much money it owes to each seller. When Amazon decides to remit funds to sellers, it initiates a pay-out flow from Amazon's bank account to a seller's bank account. This is the pay-out flow. The simplified pay-in and pay-out flows are shown in Figure 1.

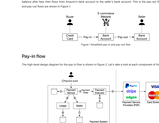


Figure 1: Pay-in flow

Pay-in flow

The high-level design diagram for pay-in flow is shown in Figure 2. Let's take a look at each component of the system.

Figure 2: Pay-in flow

Payment service

The payment service accepts payment events from users and coordinates the payment process. The first thing it usually does is a risk check for compliance, legal, and fraud concerns. If a payment event passes the risk check, the payment service has a record of the financial obligation. The payment service calls payment service provider (PSP) to move money to the seller's account. Finally, the payment service updates the payment status after the payment is processed by the payment service provider.

Payment executor

The payment executor is a single payment order via a Payment Service Provider (PSP). A payment event may contain several payment orders.

Payment Service Provider (PSP)

A PSP moves money from account X to account Y, in this simplified example. The PSP moves the money out of the buyer's credit card account.

Card schemes

Card schemes are the organizations that process credit card payments. Well-known card schemes are Visa, MasterCard, Discovery, etc. The card scheme accepted for a certain credit card is also called the card network.

Ledger

The ledger keeps a financial record of the payment transactions. For example, when a user pays the seller 5 unconsolidated from a user purchase, the payment service calls the ledger service to pay. Important in post-payment audits, such as calculating the total revenue of the e-commerce operation or reconciling balances.

Wallet

The wallet keeps the account balance of the merchants. It may also record how much a given user has paid in total.

An return in Figure 2, we pay the seller. Amazon's wallet service tracks how much money is owed to the seller.

1. The payment service receives a payment event and verifies if this payment event is generated and sent to the payment service.
2. The payment service stores the payment event in the database.
3. Sometimes, a single payment event may contain several payment orders. For example, you may select products from multiple sellers in your Amazon shopping cart. Assuming we use third-party services that support only one seller per transaction, the payment service calls the payment executor for each payment order.
4. The payment executor stores the payment order in the database.
5. The payment executor calls the external PSP to process the payment order.
6. After the payment execution has successfully processed the payment, the payment service updates the seller to record how much money is owed to the seller.
7. The wallet service returns the updated balance information to the database.
8. After the wallet service has successfully updated the seller's balance information, the payment service calls the ledger to update it.
9. The ledger service updates the ledger information to the database.

APIs for payment service

We use the RESTful API design conventions for the payment service.

POST /v1/payments

This endpoint executes a payment event. It, for instance, executes a single payment event (with multiple payment orders). The request parameters are listed below:

Field	Description	Type
buyer_info	The information of the buyer	json
checkout_id	A globally unique ID for this checkout	string
credit_card_info	This could be encrypted credit card information or a payment token. The value is PSP specific.	json
payment_orders	A list of the payment orders	list

Table 1: HTTP request parameters (example: a payment event)

The payment_orders looks like this:

Field	Description	Type
seller_account	Which seller will receive the money	string
amount	The transaction amount for the seller	string, decimal
currency	The currency for the order	string (ISO 4217 [2])
payment_order_id	A globally unique ID for the payment order	string

Table 2: payment_orders

Note that the payment_order_id is globally unique. When the payment executor sends a payment request to a third-party PSP, the payment_order_id is used to track the payment request across the payment industry.

The response body contains the status of the "accept" field to either either "accepted" (success) or "rejected" (failure).

Different processors, software, and hardware may support different numeric representations to serialization and deserialization. This difference might cause unexpected rounding errors.

The number could be corrupted by the example, Java's GSF is stored in float[10^4] per for the calendar year 2020, or extremely small the example, a number of 4.75.

It is recommended to keep numbers in string format during transmission and storage. They are only parsed to numbers when used for calculations on the client side.

GET /v1/payments/{id}

This endpoint returns the execution status of a single payment order based on payment_order_id.

The payment API documented above is similar to the API of some well-known payment processors such as Stripe, Braintree, Square, Square, and Square.

The data model for payment service

The payment service stores two tables: payment event and payment order. When we select a storage solution for a payment service, performance is usually the most important factor. Two options are the following:

- Frozen ability. Whether the single system has been used by other big financial firms for many years (for example more than 5 years) with positive feedback.
- The richness of supporting tools, such as monitoring and management tools.
- Relational vs. NoSQL database administration (DBA) can isolate. Whether we should experiment DBAs is a very important factor to consider.

Usually, we prefer a traditional relational database with ACID transaction support over NoSQL/NewSQL.

The payment event table contains detailed payment event information. Three where it looks like:

Name	Type
checkout_id	string
buyer_info	string
seller_info	string
credit_card_info	depends on the card provider
is_payment_done	boolean

Table 3: Payment event

The payment order table stores the execution status of each payment order. This is what it looks like:

Name	Type
payment_order_id	String PK
buyer_account	string
amount	string
currency	string
checkout_id	string FK
payment_order_status	string
ledger_updated	boolean
wallet_updated	boolean

Table 4: Payment order

Before we dive into the tables, let's take a look at some background information:

- The checkout_id is the foreign key. A single checkout creates a payment event that may contain several payment orders.
- The payment_order_id is a globally unique ID. We use the UUID standard. The payment service creates the UUID. In addition, the UUID is embedded in the e-commerce website's back account. The process is called dual-flow. When the payment confirmation is satisfied, such as when checkouts are delivered, the seller initiates a pay-out. Only there is the money transferred from Amazon's bank account to seller's bank account. The payment order UUID is used for pay-out. Since the pay-out flow is very far from the pay-in flow, we only need the payment order UUID to track account information.

In the payment order table (Table 4), payment_order_status is an enumerated type (enum) that keeps the execution status of the payment order. Possible values include NOT_STARTED, EXECUTING, SUCCESS, FAILED, NOT_EXECUTED. The update logic is:

- The initial status of payment_order_status is NOT_STARTED.
- When the payment executor begins to execute a payment order, the payment service, the payment_order_status is EXECUTING.
- The payment service updates the payment_order_status to SUCCESS or FAILED depending on the response of the payment executor.

Once the payment_order_status is SUCCESS, the payment service calls the wallet service to update the seller balance and updates the wallet_updated field. Only when the wallet's the balance has been updated, the payment service updates the ledger.

Since it is done, the next step for the payment service is to call the ledger service to update the ledger database by updating the ledger_updated field.

When all payment orders under the same checkout id are processed successfully, the payment service updates the is_payment_done to true in the payment event table. The payment service then sends payment status to the payment event handler. The payment event handler is similar as alert and notification system and sends out notification to the buyer about the payment status.

Double-entry ledger system

There is a very important design principle in the ledger system: the double-entry principle (also called double-entry accounting). Double-entry accounting means that every transaction is recorded in two accounts, one debit and one credit. The sum of all debits and every payment transaction in the ledgers ledger accounts add to the same amount. The account is debited and the other is credited with the same amount (Table 5).

Account	Debit	Credit
buyer	$1	
seller		$1

Table 5: Double entry system

The double entry system states that the such of the transaction action should be recorded in 2 this sort not means contrasts data gets a crash. It provides end-to-end traceability and ensures consistency throughout the system. To find out more about implementing the double-entry system, see Accounting entries using double-entry accounting Wikipedia article [3].

Hosted payment page

Most companies prefer not to deal with credit card information internally because if they do, they have to deal with complex regulations such as Payment Card Industry Data Security Standard (PCI DSS) [4]. Instead, they prefer to use a hosted payment page provided by third-party payment processors. For example, Braintree has a JavaScript SDK for hosted payment page. Stripe has a pre-built page that supports Apple Pay, Google Pay, Alipay, and a number of other standard payment card transfer requests. The point here is that the PSP provides a hosted payment page that replaces the customer card information directly, rather than relying on the payment service.

- The client sends a request to the PSP's payment service with the payment order, detailed information about the payment request from the PSP's backend. The payment service returns a URL to the client.
- The client uses the URL to load the payment page. The payment page is hosted by the PSP. Typically, it asks the payment to enter credit card information and authorize the payment. Everything is in a secure sandbox that isolates sensitive information.
- The client's browser sends the payment information directly to the PSP. The payment information never touches the client's or payment service's servers. This is a very important design. It means we don't have to store sensitive information and comply with many regulations. The client's browser receives a response from the PSP. The response is usually a redirect. The redirect URL contains the payment status and a payment token. The payment token is used to identify the payment in the payment service's database. The payment service uses the payment token to query the payment status. The client's browser then redirects to the payment service's domain. The payment service displays the payment UI, collects payment confirmation, and calls the PSP directly to complete the payment. Depending on the payment information is collected by Stripe, it even reaches our payment system. The hosted payment page usually needs two pieces of information:
  - The client secret: Stripe uses it to access the PSP's payment order under the hosted payment page. It contains enough information to retrieve the payment information from the PSP's backend.
  - The return URL: Stripe needs to know which URL to redirect to after the payment is completed. Typically, it is a URL pointed to a complete. When the PSP's payment processor finishes the payment, it redirects the browser to the return URL. Usually, the return URL is an e-commerce web page that shows the status of the checkout. Note that the redirect URL is different from the redirect [5], URL in step 3.

Figure 3: Hosted payment page

We combined the payment executor, ledger, and wallet in Figure 3 for simplicity. The payment service orchestrates the whole payment process.

The case using the "Redirect" button in the client browser. The client calls the payment service with the payment order information.

After receiving the payment information, the payment service stores payment information and calls the PSP's payment service. The payment service generates a URL and sends it to the client's browser. The client's browser calls the payment request, and the redirect URL. The payment service displays the payment UI, collects payment information, and calls the PSP directly to complete the payment. Finally, the client's browser receives the payment status.

The PSP returns a token back to the payment service. It takes a UUID or the PSP URL that uniquely identifies the payment information. The payment service uses the token to query the payment information from the PSP's backend.

The payment service sends the token to the database before calling the PSP's payment service.

Since this token is persisted, the client displays a PSP-hosted payment page. Mobile applications usually use the PSP's SDK to invoke payment UI. Web browsers typically use a redirect or a pop-up window. The payment UI is hosted by the PSP. The payment UI collects payment information and displays the payment to collect. Sensitive payment information, such as the PSP directly to complete the payment. Sensitive payment information is collected by Stripe. It never reaches our payment system. The hosted payment page usually needs two pieces of information:

- The client secret: Stripe uses it to the PSP's payment order under the hosted payment page. It contains enough information to retrieve the payment information from the PSP's backend.
- The return URL: Stripe needs to know which URL to redirect to after the payment is completed. Typically, it is a URL pointed to a complete. When the PSP's payment processor finishes the payment, it redirects the browser to the return URL. Usually, the return URL is an e-commerce web page that shows the status of the checkout. Note that the redirect URL is different from the redirect [5], URL in step 3.

Figure 4: Hosted payment page by Stripe

Figure 4 shows the hosted payment page of Stripe. It contains the credit card number, holder's name, expiration date, etc. Then, then the pay button. The PSP hosts the payment processing.

The PSP returns the payment status.

The payment service stores the payment status in the database.

The payment service calls the payment service to update the payment status.

Asynchronously, the PSP calls the payment service to update the payment status on the database. The webhook is an URL on the payment service that the PSP can use to communicate payment status. Usually, the payment service registers a webhook with the PSP when it calls the payment service for the first time. Asynchronously, the PSP calls the payment service to update the payment status. The payment service then calls the payment service to update the payment status.

So far, we assumed the hyper-path of the hosted payment page or widgets, the network connection could be unreliable and all things above could fail. Is there any synchronous way to handle failure cases? The answer is reconciliation.

Reconciliation

When system components communicate asynchronously, there is no guarantee that a message will be delivered, or a response will be received. The is synchronous in the payment operations. Asynchronous communication is increase system performance. Financial systems, such as PSP, payment, payment processor, communication as well. So how can we ensure correctness in this case?

The answer is reconciliation. It is a process that periodically compares the state among related services in order to verify they are in agreement. It is the last line of defense in the payment system.

We recommend that every payment system should run reconciliation. The addressed the correctness the balance of the back account, together with all the transactions that are the last seven or so days that account is run. The reconciliation process is usually done by a different file and compares the details with the ledger system. Figure 5 below shows where the reconciliation process fits in the system.

Figure 5: Reconciliation

Reconciliation is also used to verify that the payment system is internally consistent. For example, the states in the ledger and wallet might diverge and we could use the reconciliation system to detect any discrepancy.

It's recommended that reconciliation is accomplish, as usually as it is feasible, in order to perform manual adjustments. The reconciliation and adjustments are usually checked into three categories:

- Automatic. The discrepancy is small and can be automated. In this case, we know the cause of the mismatch, how to fix it, and it is cost-effective to develop an automation for automatic adjustment. Example: cent rounding both the internal reconciliation and adjustment.
- The reconciliation is classified, but we are unable to automate the adjustment. In this case, we know the cause of the mismatch and can fix it manually. Example: we know there is a bug causing the wrong amount to be logged. The mismatch is rare and is quick to fix. The mismatch lasts than five internal transactions.
- The mismatch is unclassified. It is more rare than the other two mismatch happens. The mismatch is up for a special investigation. The process ends up in a manual adjustment.

Handling payment processing delays

As discussed previously, an end-to-end payment request flows through many components and involves both internal and external services. While it requires several seconds to complete a payment request, the entire process should not take more than a few minutes. Most of the time, it should be a matter of seconds. Sometimes take hours or days before a payment request could take longer time.

- The PSP delays a payment request and requires an extra check or review.
- A credit card requires extra protocols like 3D Secure Authentication [7] which requires extra details from a card holder to verify a transaction.

The payment service must be able to handle these payment requests that take a long time to process. If the top stage is hosted by an third-party service, the payment service relies on the third-party payment processor to notify payment status changes.

- The PSP and card schemes operating place a cut-off time for payment cutoff time in the year. Our client should provide payment for the customer to check the current payment status.
- The PSP provides a reconciliation file on a daily basis. We can run reconciliation between our system and the payment service regulation and the PSP.

When the payment request is finally completed, the PSP calls the registered webhook mentioned above. The payment service updates its internal current and completed the request to the customer.

Alternatively, instead of updating the payment service via a webhook, some PSP must poll the location on the payment service to ask the PSP for status updates on outstanding payments.

Communication among internal services

There are two types of communication patterns that internal services use to communicate: synchronous vs asynchronous. Both are essential items.

Synchronous communication

Synchronous communication over the HTTP works well for simple calls. Examples include getting blockchain data or the code for bitcoin. It works a long request and response cycle that depends on external services. The drawbacks of this approach are:

- Low performance. It is one of the services in the chain doesn't perform well, the whole system is impacted.
- Poor availability. Failure in one service in the chain leads to the whole system being unavailable.
- Tight coupling. The request sender needs to know the receiver's address.
- Hard to scale. Without using a proxy or a service mesh, it is difficult to add the system to support a sudden increase in traffic.

Asynchronous communication

Asynchronous communication can be divided into two categories:

- Single receiver: each message is processed by one consumer or service. It's usually implemented via a shared message queue. The message queue can have multiple subscribers, but once a message is processed, it gets removed from the queue. Let's consider a payment system. Our system needs to deal with many time-consuming tasks, such as reconciliation. Tasks are consumed by various A and service B respectively. Both messages are removed from the queue as shown in Figure 6.

Figure 7: Message queue

Figure 6: Single receiver for each message

- Multiple receivers: each request (message) is processed by multiple receivers or services. Kafka works well here. When consumers subscribe to a topic, they can receive all messages in that topic. Let's consider a payment system. Our system needs to deal with many tasks, such as sending push notifications, updating financial reporting, analytics, etc. An example is illustrated in Figure 8. Payment events are published to Kafka and consumed by different services such as reconciliation system, analytics service, and billing system.

Figure 8: Multiple receivers for each message

Generally speaking, synchronous communication is simpler in design, but it doesn't affect services to be autonomous. As the dependency grows, it becomes a bottleneck for latency and scalability. Asynchronous communication is more complex in design, but it is more scalable and fault tolerant. It is a better choice for payment system. Asynchronous communication can be either single receiver or multiple receivers. Single receiver is ideal for time-consuming tasks, such as reconciliation. Multiple receivers is ideal for a payment event that is consumed by various services, such as sending push notifications, updating financial reporting, analytics, etc. An example is illustrated in Figure 8. Payment events are published to Kafka and consumed by different services such as reconciliation system, analytics service, and billing system.

asynchronous communication is a better choice.

Handling failed payments

Every payment system has to handle failed transactions. Reliability and fault tolerance are key requirements. We review some of the techniques for tackling those challenges.

Tracking payment state

Having a definitive payment state at any stage of the payment cycle is crucial. Whenever a failure happens, we can determine the current state of a payment transaction and whether a retry or refund is needed. The payment state can be persisted in an *append-only* database table.

Retry queue and dead letter queue

To gracefully handle failures, we utilize the retry queue and dead letter queue, as shown in Figure 10.

- Retry queue: retryable errors such as transient errors are routed to a retry queue.
- Dead letter queue (14): if a message fails, it is eventually moved to the dead letter queue. A dead letter queue is useful for debugging and isolating problematic messages for inspection to determine why they were not processed successfully.

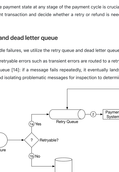


Figure 10 Handle failed payments

1. Check whether the failure is retryable.
   1a. Retryable failures are routed to a retry queue.
   1b. Non-retryable failures such as invalid input, errors are stored in a database.
2. The payment system consumes events from the retry queue and retries failed payment transactions.
3. If the payment transaction fails again:
   3a. If the retry count doesn't exceed the threshold, the event is routed to the retry queue.
   3b. If the retry count exceeds the threshold, the event is put in the dead letter queue. Those failed events might need to be investigated.

If you are interested in a real-world example of using those queues, take a look at Uber's payment system that utilizes Kafka to meet the reliability and fault-tolerance requirements [16].

Exactly-once delivery

One of the most serious problems a payment system can have is to double charge a customer. It is important to guarantee in our design that the payment system executes a payment order exactly-once [18].

At first glance, exactly-once delivery seems very hard to tackle, but if we divide the problem into two parts, it is much easier to solve. Mathematically, an operation is executed exactly-once if:

1. It is executed at least-once.
2. At the same time, it is executed at-most-once.

We will explain how to implement at-least-once using retry, and at-most-once using idempotency check.

Retry

Occasionally, we need to retry a payment transaction due to network errors or timeout. Retry provides the at least-once guarantee. For example, as shown in Figure 11, where the client tries to make a $10 payment, but the payment request keeps failing due to a poor network connection. In this example, the network eventually recovered and the request succeeded at the fourth attempt.

Figure 11 Retry

Deciding the appropriate time intervals between retries is important. Here are a few common retry strategies.

- Immediate retry: client immediately resends a request.
- Fixed intervals: wait a fixed amount of time between the time of the failed payment and a new retry attempt.
- Incremental intervals: client waits for a short time for the first retry, and then incrementally increases the time for subsequent retries.
- Exponential backoff [17]: double the waiting time between retries after each failed retry. For example, when a request fails for the first time, we retry after 1 second, if it fails a second time, we wait 2 seconds before the next retry, if it fails a third time, we wait 4 seconds before another retry.
- Cancel: the client can cancel the request. This is a common practice when the failure is permanent or repeated requests are unlikely to be successful.

Determining the appropriate retry strategy is difficult. There is no "one size fits all" solution. As a general guideline, use exponential backoff if the network issue is unlikely to be resolved in a short amount of time. Overly aggressive retry strategies waste computing resources and can cause service overload. A good practice is to provide an error code with a Retry-After header.

A potential problem of retrying is double payments. Let us take a look at two scenarios.

Scenario 1: The payment system integrates with PSP using a hosted payment page, and the client clicks the pay button twice.

Scenario 2: The payment is successfully processed by the PSP, but the response fails to reach our payment system due to network errors. The user clicks the "pay" button again or the client retries the payment.

In order to avoid double payment, the payment has to be executed at-most-once. This at-most-once guarantee is also called idempotency.

Idempotency

Idempotency is key to ensuring the at-most-once guarantee. According to Wikipedia, *"idempotence is the property of certain operations in mathematics and computer science whereby they can be applied multiple times without changing the result beyond the initial application"* [18]. From an API standpoint, idempotency means clients can make the same call repeatedly and produce the same result.

For communication between clients (web and mobile applications) and servers, an idempotency key is usually a unique value that is generated by the client and expires after a certain period of time. A UUID is commonly used as an idempotency key and it is recommended by many tech companies such as Stripe [19] and PayPal [20]. To perform an idempotent payment request, an idempotency key is added to the HTTP header: *<idempotency-key: key_value>*.

Now that we understand the basics of idempotency, let's take a look at how it helps to solve the double payment issues mentioned above.

Scenario 1: what if a customer clicks the "pay" button quickly twice?

In Figure 12, when a user clicks "pay" an idempotency key is sent to the payment system as part of the HTTP request. In an e-commerce website, the idempotency key is usually the ID of the shopping cart right before the checkout.

For the second request, it's treated as a retry because the payment system has already seen the idempotency key. When we include a previously specified idempotency key in the request header, the payment system returns the latest status of the previous request.

Figure 12 Idempotency

If multiple concurrent requests are detected with the same idempotency key, only one request is processed and the others receive the "429 Too Many Requests" status code.

To support idempotency, we can use the database's unique key constraint. For example, the primary key of the database table is served as the idempotency key. Here is how it works:

1. When the payment system receives a payment, it tries to insert a row into the database table.
2. A successful insertion means we have not seen this payment request before.
3. If the insertion fails because the same primary key already exists, it means we have seen this payment request before. The second request will not be processed.

Scenario 2: The payment is successfully processed by the PSP, but the response fails to reach our payment system due to network errors. Then the user clicks the "pay" again.

As shown in Figure 4 (step 2 and step 3), the payment service sends the PSP a nonce and the PSP returns a corresponding token. The nonce uniquely represents the payment order, and the token uniquely maps to the nonce. Therefore, the token uniquely maps to the payment order.

When the user clicks the "pay" button again, the payment order is the same, so the token sent to the PSP is the same. Because the token is used as the idempotency key on the PSP side, it is able to identify the duplicate payment and return the status of the previous execution.

Consistency

Several stateful services are called in a payment execution:

1. The payment service keeps payment-related data such as nonce, token, payment order, execution status, etc.
2. The ledger keeps all accounting data.
3. The wallet keeps the account balance of the merchant.
4. The PSP keeps the payment execution status.
5. Data might be replicated among different database replicas to increase reliability.

In a distributed environment, the communication between any two services can fail, causing data inconsistency. Let's take a look at some techniques to resolve data inconsistency in a payment system.

To maintain data consistency between internal services, ensuring exactly-once processing is very important.

To maintain data consistency between the internal service and external service (PSP), we usually rely on idempotency and reconciliation. If the external service supports idempotency, we should use the same idempotency key for payment retry operations. Even if an external service supports idempotent API, reconciliation is still needed because we shouldn't assume the external system is always right.

If data is replicated, replication lag could cause inconsistent data between the primary database and the replicas. There are generally two options to solve this:

1. Serve both reads and writes from the primary database only. This approach is easy to set up, but the obvious drawback is scalability. Replicas are used to ensure data reliability, but they don't serve any traffic, which wastes resources.
2. Ensure all replicas are always in-sync. We could use consensus algorithms such as Paxos [21] and Raft [22], or use consensus-based distributed databases such as YugabyteDB [23] or CockroachDB [24].

Payment security

Payment security is very important. In the final part of this system design, we briefly cover a few techniques for combating cyberattacks and card thefts.

Problem	Solution
Request/response eavesdropping	Use HTTPS
Data tampering	Enforce encryption and integrity monitoring
Man-in-the-middle attack	Use SSL with certificate pinning
Data loss	Database replication across multiple regions and take snapshots of data
Distributed denial-of-service attack (DDoS)	Rate limiting and firewall [25]
Card theft	Tokenization. Instead of using real card numbers, tokens are stored and used for payment
PCI compliance	PCI DSS is an information security standard for organizations that handle branded credit cards
Fraud	Address verification, card verification value (CVV), user behavior analysis, etc. [26] [27]

Table 6 Payment security

Step 4 - Wrap Up

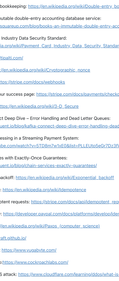
In this chapter, we investigated the pay-in flow and pay-out flow. We went into great depth about retry, idempotency, and consistency. Payment error handling and security are also covered at the end of the chapter.

A payment system is extremely complex. Even though we have covered many topics, there are still more worth mentioning. The following is a representative but not an exhaustive list of relevant topics.

- Monitoring. Monitoring key metrics is a critical part of any modern application. With extensive monitoring, we can answer questions like "What is the average acceptance rate for a specific payment method?", "What is the QPU usage of our servers?", etc. We can create and display those metrics on a dashboard.
- Alerting. When something abnormal occurs, it is important to alert on-call developers so they respond promptly.
- Debugging tools. "Why does a payment fail?" is a common question. To make debugging easier for engineers and for customer support, it is important to develop tools that allow staff to review the transaction status, processing server history, PSP records, etc. of a payment transaction.
- Currency exchange. Currency exchange is an important consideration when designing a payment system for an international user base.
- Geography. Different regions might have completely different sets of payment methods.
- Cash payment. Cash payment is very common in India, Brazil, and some other countries. Uber [28] and Airbnb [29] wrote detailed engineering blogs about how they handled cash-based payment.
- Google/Apple pay integration. Please read [30] if interested.

Congratulations on getting this far! Now give yourself a pat on the back. Good job!

Chapter Summary



Reference Materials

[1] Payment system: https://en.wikipedia.org/wiki/Payment_system

[2] ANLICFT: https://en.wikipedia.org/wiki/Moses_Jaundelow

[3] Card scheme: https://en.wikipedia.org/wiki/Card_scheme

[4] ISO 4217: https://en.wikipedia.org/wiki/ISO_4217

[5] Stripe API Reference: https://stripe.com/docs/api

[6] Double-entry bookkeeping: https://en.wikipedia.org/wiki/Double-entry_bookkeeping

[7] Books, an immutable double-entry accounting database service: https://developer.square.com/blog/books-an-immutable-double-entry-accounting-database-service/

[8] Payment Card Industry Data Security Standard: https://en.wikipedia.org/wiki/Payment_Card_Industry_Data_Security_Standard

[9] Tokₑₙ: https://tokenex.com/

[10] Nonce: https://en.wikipedia.org/wiki/Cryptographic_nonce

[11] Webhooks: https://stripe.com/docs/webhooks

[12] Customize your success page: https://stripe.com/docs/payments/checkout/custom-success-page

[13] 3D Secure: https://en.wikipedia.org/wiki/3-D_Secure

[14] Kafka Connect Deep Dive – Error Handling and Dead Letter Queues: https://www.confluent.io/blog/kafka-connect-deep-dive-error-handling-dead-letter-queues/

[15] Reliable Processing in a Streaming Payment System: https://www.youtube.com/watch?v=5TDbm7e7oE0&list=PLLE0Jo5eOi7DLSMQUuGGSkJ_W4Jw-RJ

[16] Chain Services with Exactly-Once Guarantees: https://www.confluent.io/blog/chain-services-exactly-guarantees/

[17] Exponential backoff: https://en.wikipedia.org/wiki/Exponential_backoff

[18] Idempotence: https://en.wikipedia.org/wiki/Idempotence

[19] Stripe idempotent requests: https://stripe.com/docs/api/idempotent_requests

[20] Idempotency: https://developer.paypal.com/docs/platforms/developer/idempotency/

[21] Paxos: https://en.wikipedia.org/wiki/Paxos_(computer_science)

[22] Raft: https://raft.github.io/

[23] YugabyteDB: https://www.yugabyte.com/

[24] CockroachDB:https://www.cockroachlabs.com/

[25] What is DDoS attack: https://www.cloudflare.com/learning/ddos/what-is-a-ddos-attack/

[26] How Payment Gateways Can Detect and Prevent Online Fraud: https://www.chargebee.com/blog/prevent-online-billing-pays-online-fraud/

[27] Advanced Technologies for Detecting and Preventing Fraud at Uber: https://www.uber.com/en-PK/blog/project-radar-intelligent-early-fraud-detection/

[28] Re-Architecting Cash and Digital Wallet Payments for India with Uber Engineering: https://eng.uber.com/india-payments/

[29] Scaling Airbnb's Payment Platform: https://medium.com/airbnb-engineering/scaling-airbnbs-payment-platform-43ebfc99b324

[30] Payments Integration at Uber: A Case Study: https://www.youtube.com/watch?v=xoQG3BG0BA

Mark as Complete

Copyright ©2022-2026 BytebyteGo Inc. All rights reserved.

Ask AI